CHATGPT MARKET & SUBSCRIPTION ANALYTICS

India vs USA | Python • PostgreSQL • Power BI

Project documentation and beginner-friendly HR interview guide

Project component	What it does
Python / Pandas	Inspect, clean, flag data-quality issues, and combine two simulated sales CSVs.
PostgreSQL / SQL	Query transaction data to answer business questions.
Power BI	Present KPIs, comparisons, customer/subscription patterns, and public usage trends.

Important data note: India and USA transaction files are simulated sales data. Page 4 uses separate public OpenAI Signals CSVs. The simulated sales data must not be described as real OpenAI customer or revenue records.

Prepared as a learning guide from the project details discussed. Verify code and exact SQL queries against your saved project files before claiming every example was executed.

1. Project overview

Project title: ChatGPT Market & Subscription Analytics: India vs USA

Objective: Build a small analytics workflow from raw CSV files to cleaned data, SQL analysis, and an interactive Power BI report. The report is designed to demonstrate how a data analyst can prepare data, calculate useful metrics, compare markets, and communicate findings visually.

Business questions

• How many transactions were paid, failed, or refunded?

• What is the successful revenue and average paid transaction value in each market?

• Which subscription plans contribute the most successful revenue?

• How do transaction status and monthly revenue vary between India and the USA?

• What patterns appear across customer segments, subscription plans, billing cycles, and cancellation events?

• What trends are visible in public OpenAI Signals message-share datasets?

Tools used

Tool	Role in the project
Python + Pandas	Read CSV files, inspect columns, check data quality, combine datasets, and prepare summaries/plots.
PostgreSQL + pgAdmin	Store/query the combined dataset and answer business questions with SQL.
Power BI Desktop	Create KPI cards, charts, slicers, and four report pages.
CSV files	Provide simulated transaction data and separate public usage-share datasets.

2. Data sources and folder structure

The project uses two different kinds of data. Keep their purpose separate when presenting the project.

A. Simulated transaction data

```
data/raw/chatgpt_india_simulated_sales_25000.csv
data/raw/chatgpt_usa_simulated_sales_25000.csv

data/cleaned/chatgpt_india_simulated_sales_25000_cleaned.csv
data/cleaned/chatgpt_usa_simulated_sales_25000_cleaned.csv
data/cleaned/combined_sales_50000.csv
```

The two input files contain 25,000 rows each. They were combined into a 50,000-row analysis file. These are fictional/simulated sales records created for portfolio practice—not actual OpenAI transactions.

B. Public OpenAI Signals data

```
data/raw/public_release_csv/
    share_of_messages_by_topic_month.csv
    global_share_of_messages_by_gender_month.csv
    share_of_messages_by_age_group_month.csv
    ...other public release CSV files
```

These files contain published message-share statistics by dimensions such as topic, age-group category, gender category, country, and month/quarter. They do not provide private individual ChatGPT histories. Use only the metric definitions and columns actually present in each CSV.

3. Step-by-step Python workflow

Step 1 — Read the raw CSV files

Pandas is a Python library for working with table-shaped data. read_csv() loads a CSV into a DataFrame, which behaves like a table.

```
import pandas as pd

india = pd.read_csv(
    "data/raw/chatgpt_india_simulated_sales_25000.csv"
)
usa = pd.read_csv(
    "data/raw/chatgpt_usa_simulated_sales_25000.csv"
)

print(india.shape)
print(usa.shape)
print(india.head())
print(india.info())
```

How to explain: “I loaded the India and USA CSV files using Pandas, then checked their row counts, column names, sample records, and data types before cleaning.”

Step 2 — Check data quality

```
# Missing values in each column
print(india.isna().sum())

# Exact duplicate rows
print(india.duplicated().sum())

# Duplicate transaction IDs
print(india["transaction_id"].duplicated().sum())

# Inspect column data types
print(india.dtypes)
```

The project checks for missing values, duplicate rows, duplicate transaction IDs, and unexpected data types. The reported review found no missing values, no duplicate rows, and no duplicate transaction IDs in the combined analysis workflow. If you rerun the code, use the new output as the final evidence.

Step 3 — Review unusual values rather than deleting them automatically

• Negative net revenue can represent refunds; it should not automatically be treated as an error.

• Failed transactions have zero revenue in this dataset.

• Cancellation-flag mismatches were flagged for review instead of being silently corrected.

• A data-quality flag means “check this record”; it does not automatically prove the source record is wrong.

The recorded mismatch counts were 496 in the India data and 448 in the USA data. These should be explained as flagged inconsistencies in the simulated dataset.

Step 4 — Combine India and USA data

```
# Add the two datasets row-wise, one below the other
combined = pd.concat(
    [india, usa],
    ignore_index=True
)

print(combined.shape)
combined.to_csv(
    "data/cleaned/combined_sales_50000.csv",
    index=False
)
```

How to explain: “The two files represented the same type of transaction data for two markets, so I used Pandas concat to stack their rows into one table. The country column lets me filter and compare the markets later.”

Important: concat is not the same as a SQL JOIN. Concatenation stacks rows; a join matches related rows across tables using a key.

Step 5 — Create analysis-ready fields and summaries

The dataset includes fields such as country, subscription_plan, billing_cycle, transaction_date/month, payment_status, net_revenue_local, customer_segment, subscription_event, and flags. Before creating metrics, check whether fields are correctly typed and whether status values are consistent.

```
# Example: inspect payment status counts
print(combined["payment_status"].value_counts(dropna=False))

# Example: inspect country and status together
print(pd.crosstab(
    combined["country"],
    combined["payment_status"]
))
```

These are examples of checks suitable for this workflow. Use your saved script as the source of truth for which exact preprocessing steps were run.

4. PostgreSQL and SQL analysis

The combined CSV was imported into PostgreSQL, with the analysis table referred to as combined_sales in the database workflow. In Power BI, the imported table appears as combined_sales_50000. A database table stores rows and columns so queries can filter, group, and summarize records.

Example 1 — Filter paid transactions

```
SELECT *
FROM combined_sales
WHERE payment_status = 'Paid';
```

WHERE filters individual rows before grouping. This query returns only transactions with Paid status.

Example 2 — Count transactions by payment status

```
SELECT
    payment_status,
    COUNT(*) AS transaction_count
FROM combined_sales
GROUP BY payment_status
ORDER BY transaction_count DESC;
```

COUNT(*) counts rows; GROUP BY creates one result group per status; ORDER BY sorts the result.

Example 3 — Successful revenue by country

```
SELECT
    country,
    SUM(net_revenue_local) AS successful_revenue_local
FROM combined_sales
WHERE payment_status = 'Paid'
GROUP BY country;
```

This is useful for market-specific revenue summaries, but local currencies must not be added together or compared as if they were the same unit. India is in INR and USA is in USD.

Example 4 — HAVING after aggregation

```
SELECT
    country,
    COUNT(*) AS paid_transactions
FROM combined_sales
WHERE payment_status = 'Paid'
GROUP BY country
HAVING COUNT(*) > 20000;
```

WHERE filters rows before grouping. HAVING filters groups after aggregation. HAVING does not mean ORDER BY.

Example 5 — CASE to label revenue

```
SELECT
    transaction_id,
    net_revenue_local,
    CASE
```

```
        WHEN net_revenue_local > 40000 THEN 'High'
        WHEN net_revenue_local >= 5000 THEN 'Medium'
        ELSE 'Low'
    END AS revenue_category
FROM combined_sales;
```

CASE applies conditional logic and creates a readable category for each row. The thresholds are illustrative and should match the actual saved query if you used different rules.

What about SQL JOIN?

A JOIN combines columns from related tables by matching a key, for example customer_id. The project's main data preparation step described here combined two country files with Pandas concat; that operation is not a join. Do not tell an interviewer that you joined two SQL tables in this project unless your saved SQL file contains an actual JOIN that you ran.

```
-- Generic JOIN example only (requires these two tables to exist)
SELECT
    c.customer_id,
    c.customer_segment,
    o.transaction_id
FROM customers AS c
JOIN orders AS o
    ON c.customer_id = o.customer_id;
```

This is a learning example, not a claim that these exact tables were part of the implemented project.

5. Metrics and interpretation

Metric	Meaning	Careful interpretation
Transactions	Number of transaction rows	Not necessarily the number of unique customers.
Paid transactions	Rows whose payment_status is Paid	Used as successful transactions in this project.
Failed transactions	Rows whose status is Failed	Failed transactions have zero revenue in the simulated data.
Refunded transactions	Rows whose status is Refunded	Refunds should be analyzed separately from successful paid revenue.
Successful revenue	Sum of net revenue for Paid rows	Keep INR and USD separate unless applying a clearly disclosed conversion
Unique customers	Distinct customer_id count	A customer can have multiple transactions.
Cancellation events	Rows with subscription_event = cancellation	Events are not necessarily unique customers who cancelled.
Cancellation rate	Cancellation events / total transactions	Label it as an event-based rate, not a verified customer churn rate.

Known figures from the simulated dataset

Metric	India	USA
Total transactions	25,000	25,000
Paid	23,679	23,784
Failed	1,040	961
Refunded	281	255
Successful revenue	■145,974,943.40	$1,826,964.90
Average paid transaction	■6,164.74	$76.81
Payment success rate	94.72%	95.14%

These values are from the discussed simulated dataset summary. They are not actual OpenAI financial results.

6. Power BI report — four pages

Power BI turns prepared data into an interactive report. A KPI card shows one important number; a chart compares categories or shows change over time; a slicer lets a viewer filter the report.

Page 1 — Executive Overview

• KPI cards summarize successful transactions, successful revenue, and growth indicators.

• Monthly Transaction Trends shows how transaction activity changes by month.

• Payment Status Distribution compares paid, failed, and refunded transactions.

• Successful Revenue by Subscription Plan compares paid revenue across plans.

• Payment Method Distribution shows the transaction mix by payment method.

• Geographic Distribution compares available geographic categories.

Interview explanation: “The Executive Overview is the landing page. It gives a quick summary of transaction performance, payment outcomes, revenue by plan, payment methods, and geography.”

Page 2 — India vs USA

• India Revenue and USA Revenue cards show paid revenue in each market’s local currency.

• India Success Rate and USA Success Rate cards show paid transactions divided by total transactions for each country.

• Payment Status Comparison uses a clustered column chart to compare Paid, Failed, and Refunded counts by country.

• Subscription Plan Comparison uses a clustered bar chart to compare transaction counts by plan and country; it counts transactions, not unique subscribers.

• India Monthly Revenue Trend shows India paid revenue over time, with a crore conversion where configured.

• USA Monthly Revenue Trend shows USA paid revenue over time in USD.

• Transaction Volume by Country compares transaction-row counts.

Interview explanation: “This page compares two markets side by side. I keep the currencies separate because INR and USD are not directly comparable. The success-rate cards make the payment outcome comparison easier to understand.”

Page 3 — Customer & Subscription

• Total Unique Customers uses a distinct count of customer_id.

• New Customers counts distinct customers marked is_new_customer = Yes.

• Cancelled Transactions counts rows where subscription_event = cancellation.

• Cancellation Rate is the cancellation-event count divided by total transaction count in this implementation.

• Customers by Segment uses a clustered bar chart with distinct customer count by customer_segment.

• Subscription Plan Distribution uses distinct customer count by subscription_plan; a customer may appear in more than one plan if records indicate that.

• Billing Cycle Distribution uses a donut chart to show transaction count by billing_cycle.

• Cancellations by Subscription Plan uses a bar chart filtered to cancellation events.

Interview explanation: “This page explores customer segments and subscription behavior. I use distinct customer counts where I want to count people rather than transaction rows, and I distinguish cancellation events from unique

customers who cancelled.”

Page 4 — Real ChatGPT Usage Trends

This page uses public OpenAI Signals CSVs, not the simulated sales table. The imported files discussed include:

• share_of_messages_by_topic_month.csv — month, topic, share_of_messages.

• global_share_of_messages_by_gender_month.csv — month, typical_name_gender, share_of_messages.

• share_of_messages_by_age_group_month.csv — month, age_group, share_of_messages.

Suggested visuals are a line chart for topic shares over time, a gender-category visual over time, and a line chart for age-group shares over time. The actual visual types should match the charts you finish building. The share field is a proportion/share metric, not a count of messages. Avoid interpreting these categories as verified identities of individual users.

Interview explanation: “The final page is separate from the sales analysis. I used public OpenAI Signals data to visualize published message-share patterns by topic, gender category, and age group over time. It is a separate dataset and does not expose private individual conversations or actual subscription revenue.”

7. Power BI DAX measures

DAX is the formula language used in Power BI to create calculations. Below are the core examples that match the metrics discussed.

Total unique customers

```
Total Unique Customers =
DISTINCTCOUNT(combined_sales_50000[customer_id])
```

New customers

```
New Customers =
CALCULATE(
    DISTINCTCOUNT(combined_sales_50000[customer_id]),
    combined_sales_50000[is_new_customer] = "Yes"
)
```

Cancelled transactions

```
Cancelled Transactions =
CALCULATE(
    COUNTROWS(combined_sales_50000),
    combined_sales_50000[subscription_event] = "cancellation"
)
```

Cancellation rate (event-based)

```
Cancellation Rate =
DIVIDE(
    [Cancelled Transactions],
    [Total Transactions],
    0
)
```

CALCULATE changes the filter context for a calculation. DISTINCTCOUNT counts different IDs. COUNTROWS counts rows. DIVIDE safely handles a zero denominator. Confirm the exact column names and measure definitions in your PBIX before copying these formulas into a final technical presentation.

8. Interview-ready project explanation

One-minute answer

“My project is ChatGPT Market and Subscription Analytics, built using Python, PostgreSQL, and Power BI. I started with two simulated transaction datasets for India and the USA, each containing 25,000 rows. Using Pandas, I inspected data types, missing values, duplicates, transaction IDs, and unusual records. I reviewed refunds, failed

payments, and cancellation-flag mismatches rather than automatically deleting them, then combined the country datasets into a 50,000-row file. I used PostgreSQL queries to filter transactions, group results, calculate counts and revenue, and create business summaries. In Power BI, I built four report pages: Executive Overview, India vs USA, Customer & Subscription, and a separate page using public OpenAI Signals usage-share data. The project helped me practice the full analytics workflow from data preparation to communicating results with dashboards."

Three-to-five-minute answer structure

• Problem and objective: show how transaction and public usage data can be prepared and analyzed.

• Data: explain the simulated India/USA transaction files and the separate public Signals files.

• Python: describe loading, inspection, data-quality checks, flagging, and concatenation.

• SQL: describe filtering with WHERE, aggregation with GROUP BY, filtering groups with HAVING, and conditional labels with CASE.

• Power BI: walk through the purpose of each of the four pages and the main visuals.

• Outcome: explain the types of insights the dashboard supports and one limitation you handled carefully.

Common HR / technical questions

Q. Why did you use Python?
A. Pandas made it easier to inspect, clean, combine, and summarize CSV data reproducibly.

Q. Why combine the two country files?
A. They shared a similar structure. Stacking them created one analysis table, while the country column preserved which market each row belonged to.

Q. What is the difference between concat and JOIN?
A. Concat stacks rows from datasets with similar columns. JOIN matches related rows from tables using a key.

Q. Why did you not delete every negative revenue value?
A. Negative values can represent refunds. I checked their meaning before treating them as errors.

Q. WHERE vs HAVING?
A. WHERE filters rows before grouping; HAVING filters grouped results after aggregation.

Q. COUNT(*) vs SUM(revenue)?
A. COUNT(*) counts rows; SUM(revenue) adds revenue values.

Q. Why Power BI after Python and SQL?
A. Python and SQL prepare and analyze data; Power BI makes results interactive and easier for stakeholders to explore.

Q. Can you compare INR revenue directly with USD revenue?
A. No. The currencies differ, so I present them separately or clearly state an explicit conversion assumption.

Q. Is the sales data actual OpenAI data?
A. No. The transaction dataset is simulated for portfolio practice. The final page uses public OpenAI Signals usage-share CSVs.

Q. Does cancellation rate mean customer churn?
A. Not in this implementation. It is cancellation events divided by transactions, so I label and interpret it as an event-based rate.

9. Limitations and responsible presentation

• The India/USA sales data is simulated and should be identified as such on the report and in interviews.

• Revenue comparisons across INR and USD need currency awareness. If a fixed exchange-rate assumption was used for a combined USD-equivalent measure, disclose the rate and say it is illustrative, not live FX.

• Transaction counts are not the same as unique subscribers or customers.

• Cancellation-event counts are not automatically the number of unique cancelled customers.

• Public message-share data is not the same as message volume, user count, or private conversation content.

• Do not claim to have used SQL JOINs in the implemented workflow unless your saved SQL script contains a JOIN query that you actually ran.

10. Final checklist before presenting

• Open the Python script and confirm the exact cleaning and concatenation steps.

• Open the saved SQL file and practice explaining each query in your own words.

• Open the Power BI file and verify all four pages, chart titles, slicers, and navigation buttons.

• Confirm each chart uses the correct aggregation and table.

• Be ready to explain one data-quality issue and why you flagged or retained it.

• Use the one-minute answer as a starting point, then practice speaking naturally rather than memorizing every sentence.

End of guide. This document is an interview learning aid based on the project details shared in the conversation. For final submission, compare the examples with your actual Python scripts, SQL file, and Power BI report.